

# 601.433/633 Intro Algorithms

## Recitation 3

Fall 2026

### 1 Array Doubling

An array starts with capacity 1. When it becomes full, allocate a new array of twice the capacity and copy all existing elements. Show that a sequence of  $n$  insertions takes  $O(n)$  total time, meaning that the amortized cost of insertion is  $O(1)$ , even though a single insertion can cost  $\Theta(n)$ .

Give proofs using:

- (a) direct analysis;
- (b) the piggy bank method;
- (c) the potential method.

**Solution:** Assume that writing a newly inserted element costs 1, and copying an existing element costs 1.

**(a) Direct analysis:** Resizes occur when the old capacities are

$$1, 2, 4, 8, \dots$$

If the final number of elements is  $n$ , the total cost of copying elements is at most

$$1 + 2 + 4 + \dots + 2^{\lfloor \log_2 n \rfloor} < 2n.$$

The  $n$  ordinary insertions cost another  $n$  to write, so the total cost of  $n$  insertions is  $O(n)$ . Hence the amortized insertion cost is  $O(1)$ .

**(b) Piggy bank method:** We charge ourselves 3 units for each insertion. We use 1 unit to pay for writing the new element and save 2 units as credit. Clearly, the bank is never negative: After a doubling that leaves the array with capacity  $m$ , exactly  $\frac{m}{2}$  new insertions occur that have not been copied earlier, before the array size is doubled from capacity  $m$  to capacity  $2m$ . Those  $\frac{m}{2}$  insertions contribute  $m$  units of saved credit which is enough to pay for the  $m$ -element copy at the next resize (to capacity  $2m$ ).

We now calculate the amortized cost of insertion. If an insertion does not cause resizing, then the actual cost  $c$  of the insertion is 1, while the change in the bank  $\Delta L$  is 2, giving us an amortized cost

$$c' = c + \Delta L = 3.$$

On the other hand, if an insertion happens when the array is at full capacity  $m$ , then after doubling we have an array of size  $2m$  that we need to copy the existing  $m$  elements into. We pay for copying using our saved credit in the bank. Thus, the actual cost of copying  $m$  elements and adding a new one is  $c = m + 1$ , while the change in the bank is

$$\Delta L = (3 - 1) - m = 2 - m,$$

again giving us amortized cost

$$c' = c + \Delta L = (m + 1) + (2 - m) = 3.$$

Therefore, the amortized cost of insertion is  $3 = O(1)$ .

**(c) Potential method:** Let  $n_t$  be the number of stored elements and  $m_t$  the capacity after an operation. Define a potential function

$$\Phi_t = 2n_t - m_t.$$

After the first insertion, the array is always at least half full, so  $\Phi_t \geq 0$ .

If an insertion does not resize, its actual cost  $c_t$  is 1, while  $n_t = n_{t-1} + 1$  and  $m_t = m_{t-1}$ , giving us

$$\begin{aligned} \Delta\Phi &= \Phi_t - \Phi_{t-1} \\ &= 2n_t - m_t - (2n_{t-1} - m_{t-1}) \\ &= 2(n_{t-1} + 1) - m_{t-1} - 2n_{t-1} + m_{t-1} \\ &= 2. \end{aligned}$$

Thus, the amortized cost is  $c'_t = c_t + \Delta\Phi = 3$ .

If the array is full with  $n_{t-1} = m_{t-1} = m$  before insertion, then after array doubling and inserting we have  $n_t = m + 1$  and capacity  $m_t = 2m$ . The actual cost  $c_t$  is  $m + 1$ , and

$$\begin{aligned} \Delta\Phi &= \Phi_t - \Phi_{t-1} \\ &= (2n_t - m_t) - (2n_{t-1} - m_{t-1}) \\ &= (2(m + 1) - 2m) - (2m - m) \\ &= 2 - m. \end{aligned}$$

Hence, the amortized cost is

$$c'_t = c_t + \Delta\Phi = (m + 1) + (2 - m) = 3.$$

Thus, every insertion has  $O(1)$  amortized cost.

## 2 Changing the growth factor

- (a) Instead of doubling, suppose that whenever an array fills, its capacity is multiplied by a fixed integer  $c > 1$ . Determine the amortized insertion cost as a function of  $c$ .
- (b) Suppose that whenever an array becomes full, its capacity increases from  $m$  to  $m + c$ . What is the amortized cost per insertion?

**Solution:**

- (a) Resizing of array happens at capacities

$$1, c, c^2, c^3, \dots$$

If the final number of elements is  $n$ , the total number of copied elements is bounded by

$$\begin{aligned} 1 + c + c^2 + \dots + c^{\lfloor \log_c n \rfloor} &= \frac{c^{\lfloor \log_c n \rfloor + 1} - 1}{c - 1} \\ &\leq \frac{c}{c - 1} n - \frac{1}{c - 1} \\ &= O\left(\frac{c}{c - 1} n\right). \end{aligned}$$

Adding the cost of writing the  $n$  ordinary insertions gives us total cost

$$O\left(n + \left(\frac{c}{c - 1}\right)n\right).$$

Thus the amortized cost per insertion is

$$\boxed{O\left(1 + \frac{c}{c - 1}\right)}.$$

For every fixed constant  $c > 1$ , this is  $O(1)$ .

- (b) After the first resize at capacity 1, a resize occurs after every  $c$  insertions. The capacities copied are

$$1, 1 + c, 1 + 2c, 1 + 3c, \dots$$

If the final number of elements is  $n$ , the total cost of copying elements is at most

$$\begin{aligned} &1 + (1 + c) + (1 + 2c) + \dots + \left(1 + c \left\lfloor \frac{n}{c} \right\rfloor\right) \\ &= \left(1 + \left\lfloor \frac{n}{c} \right\rfloor\right) + c \left(1 + 2 + \dots + \left\lfloor \frac{n}{c} \right\rfloor\right) \\ &= O(n^2). \end{aligned}$$

Thus the total cost of  $n$  insertions is  $O(n^2)$  and the amortized cost per insertion is

$$\boxed{O(n)}.$$